

category: avalonia

tags: [toast, KzToast, TrayIcon, Hide, window-lifecycle, minimize-to-tray]

severity: medium

created: 2026-07-28

updated: 2026-07-28

project-origin: App-Access-V2 (AppShellWindow — minimize to tray)

Toast (KzToastHost) là con của Window → biến mất ngay khi Window.Hide(), không kịp cho user đọc

Tình huống gặp phải

AppShellWindow bấm nút X (Close) → parity hành vi WinForms cũ: `e.Cancel = true; Hide(); trayIcon.IsVisible = true;`

— chỉ thu nhỏ xuống khay hệ thống, không thoát hẳn. User yêu cầu bổ sung thông báo cho biết việc này (để họ biết phải thao tác ở tray icon nếu muốn thoát thật).

Triệu chứng / Lỗi

Gọi `KzToast.Show(...)` ngay trước `Hide()` trong cùng handler `Closing` — về lý thuyết code chạy đúng thứ tự, nhưng nếu gọi `Hide()` ngay lập tức (không có gì chờ), toast sẽ không kịp render/hiện ra vì `KzToastHost` nằm trong visual tree của chính Window đó — Window ẩn thì toàn bộ nội dung con (gồm cả toast) ẩn theo ngay lập tức.

Nguyên nhân gốc rễ (Root Cause)

`KzToastHost` được đặt làm con cuối cùng của `Panel` gốc trong `AppShellWindow.axaml` (để nổi trên mọi nội dung khác của window). Nó không phải overlay ở cấp OS/desktop — nó chỉ là 1 control trong visual tree của window. `Window.Hide()` ẩn toàn bộ visual tree ngay lập tức (không có transition/delay mặc định), nên toast con cũng biến mất cùng lúc, trước khi user kịp đọc.

Giải pháp

Không gọi `Hide()` ngay — cho toast một khoảng thời gian ngắn để hiển thị trước, dùng

`DispatcherTimer` một lần (one-shot) rồi mới `Hide()` + bật `TrayIcon.IsVisible`:

```
Closing += (_, e) =>
{
    if (_reallyClose) return;
    e.Cancel = true;
    KzToast.Show("Ứng dụng đã thu nhỏ xuống khay hệ thống...", KzToastKind.Info,
        TimeSpan.FromSeconds(3));
}
```

```
var hideTimer = new DispatcherTimer { Interval = TimeSpan.FromMilliseconds(1200) };
hideTimer.Tick += (_, _) =>
{
    hideTimer.Stop();
    Hide();
    _trayIcon.IsVisible = true;
};
hideTimer.Start();
};
```

Delay (1.2s) ngắn hơn thời lượng toast (3s) — đủ để user đọc dòng đầu trước khi window biến mất xuống tray; không cần chờ hết animation toast.

Áp dụng lại (How to reuse)

- Bất kỳ toast/overlay nào là con của 1 Window cụ thể (không phải overlay ở cấp `Application`/OS) → nếu hành động ngay sau đó ẩn/đóng chính Window đó (`Hide()`, `Close()`, chuyển màn hình), **PHẢI** delay hành động ẩn bằng timer để toast kịp hiển thị.
- Không dùng `await Task.Delay(...)` trong event handler đồng bộ nếu handler không phải `async` — dùng `DispatcherTimer` one-shot (`Stop()` ngay trong `Tick` đầu tiên) để tránh block hoặc reentrancy.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Nếu code build với timer delay > thời lượng toast, sẽ có khoảng trống window vẫn hiện nhưng toast đã tắt trước — nên set delay **NGẮN HƠN** thời lượng toast (ở đây $1.2s < 3s$).
- ⚠ `NativeMenuItem` không có `Icon` bitmap sẵn trong project (không có asset icon rồi) → dùng prefix ký tự đơn giản (`ı`, `×`) trong `Header` thay vì bitmap; tránh emoji/pictograph phức tạp (`[[avalonia-icon-emoji-khong-doc-duoc-o-co-nho]]`) dù đây là native OS menu (không phải Avalonia font rendering) nên rủi ro thấp hơn, vẫn nên giữ đơn giản cho nhất quán.

Tham chiếu

- File: `iAccessDesktopv2.Avalonia/iAccessDesktopv2.Avalonia/Views/Shell/AppShellWindow.axaml.cs`
- Project: App-Access-V2, yêu cầu user 2026-07-28 (thông báo khi bấm Close chỉ thu nhỏ xuống tray)